FORK INFOSYSTEMS

Karad, Maharashtra

FUNDAMENTALS OF COMPUTER & MULTI O.S.

Hardware • Operating Systems • Booting • File Systems • Multi-OS

Covers	Computer fundamentals, hardware devices, disk structure, OS & kernel concepts, boot process, OS survey, partitioning, file systems, bootloaders & multi-OS setups
Format	Seminar Notes — 10 Parts, Tables & Diagrams
Prepared For	Java Full Stack Developer Course Students
Prepared By	FORK INFOSYSTEMS, Karad, Maharashtra

Contact: 7757962804 / 9172135355

Mr. Ranjit Sir

TABLE OF CONTENTS

PART 1 Fundamentals of Computer

1.1 What is a Computer?

A computer is an electronic device that accepts data as input, processes it according to a set of instructions (program), stores intermediate and final results, and produces meaningful output. Computers operate at extremely high speeds, measured in GHz (billions of cycles per second), and can perform trillions of operations per second (FLOPS). The fundamental principle has remained unchanged since Charles Babbage's Analytical Engine — only the speed and miniaturisation have changed dramatically.

Component	Role	Example
Input	Receives raw data	Keyboard, Mouse, Sensor
Processing	Executes instructions (CPU)	Intel Core i9, AMD Ryzen
Memory	Temporary fast storage	DDR5 RAM
Storage	Permanent data storage	NVMe SSD, HDD
Output	Presents result to user	Monitor, Printer, Speaker

1.2 The 5 Rules of Computing

Every computing task — from browsing the web to rendering a 3D scene — obeys these five fundamental rules:

Rule 1 — Binary Representation

All data (numbers, text, images, audio) is represented as binary (0 and 1). Transistors act as switches: ON = 1, OFF = 0. Everything is ultimately a number.

Rule 2 — Stored Program

Instructions (programs) are stored in memory alongside data. The CPU fetches, decodes, and executes them sequentially unless a branch/jump instruction changes flow.

Rule 3 — Fetch-Decode-Execute Cycle

The CPU continuously runs the FDE cycle: (a) Fetch instruction from RAM into the Instruction Register, (b) Decode it in the Control Unit, (c) Execute via the ALU or move data. Modern CPUs pipeline and superscale this cycle.

Rule 4 — Memory Hierarchy

Speed vs cost dictates a hierarchy: Registers (fastest, ~1 cycle) > L1/L2/L3 Cache (nanoseconds) > RAM (tens of ns) > SSD (microseconds) > HDD (milliseconds). The OS manages which data lives where.

Rule 5 — Input / Process / Output

Every program takes input (user, file, network), transforms it using CPU/RAM, and produces output (screen, file, network). This IPO model is universal.

1.3 Why Binary (Base-2) in Computers?

Computers use binary (base-2) because transistors — the basic building block of all digital chips — have only two stable states: conducting (1) and non-conducting (0). Using two states is extremely reliable because noise must exceed 50% of the signal to cause an error, whereas higher bases (decimal = 10 states) would require far more precise voltage levels and would be far more error-prone.

■ ***Note:*** *Common misconception: The phrase 'computers use base-8' is sometimes heard because octal (base-8) and hexadecimal (base-16) are convenient shorthand for binary — each octal digit = 3 bits, each hex digit = 4 bits. But underneath, everything is binary.*

Decimal	Binary	Octal	Hex
0	0000	0	0
7	0111	7	7
8	1000	10	8
15	1111	17	F
16	10000	20	10
255	11111111	377	FF

■ ***Note:*** *1 Byte = 8 bits. Hexadecimal is used in memory addresses, color codes, assembly, and network protocols.*

PART 2 Hardware Devices

2.1 Power Supply Unit (PSU)

The PSU converts AC mains power (110V/220V) to regulated DC voltages required by computer components: +12V (CPU, GPU, motors), +5V (logic circuits, older devices), +3.3V (modern ICs, RAM). PSU efficiency is rated by the 80 PLUS standard (Bronze, Silver, Gold, Platinum, Titanium). A higher rating means less heat and lower electricity bills.

Type / Rating	Description
ATX	Standard desktop PSU (24-pin connector)
SFX	Small form factor — for mini-ITX builds
Modular	Cables detach — better airflow, cleaner build
Wattage	Budget: 450W | Gaming: 650W | Workstation: 850W+
Protection	OVP, UVP, OCP, OTP, SCP — protect components

2.2 Motherboard (Intel / AMD / ARM)

The motherboard is the central circuit board connecting all components. It contains the chipset (manages data flow between CPU, RAM, PCIe, USB), VRM (Voltage Regulator Module for CPU power), BIOS/UEFI chip, expansion slots, and I/O ports.

Platform	Socket	Chipset Examples	Notes
Intel	LGA 1700/1851	Z790, B760, H610	Pins on motherboard
AMD	AM5 / AM4	X670, B650, A620	Pins on CPU
ARM	SoC (soldered)	Apple M-series, Qualcomm	CPU+RAM on one chip

2.3 RAM — DDR & LPDDR

RAM (Random Access Memory) is the CPU's working memory. It is volatile — data is lost on power-off. RAM speed is measured in MT/s (mega-transfers per second) and latency in CL (CAS Latency). Double Data Rate (DDR) transfers on both clock edges, doubling effective bandwidth.

Generation	Voltage	Speed Range	Primary Use
DDR3	1.5V	800-2133 MT/s	Older desktops/servers
DDR4	1.2V	2133-5333 MT/s	Desktop / Server (current)
DDR5	1.1V	4800-8400 MT/s	Latest desktops / AI workloads
LPDDR4	1.1V	3200-4267 MT/s	Mobile / thin laptops
LPDDR5	1.05V	6400-8533 MT/s	Flagship phones, M-series Macs

■ ***Note:*** *LPDDR (Low Power DDR) is soldered directly to the board to save space and power — it cannot be upgraded after purchase.*

2.4 GPU — Integrated & Discrete

A GPU (Graphics Processing Unit) contains thousands of small cores optimised for parallel floating-point operations. Originally for rendering pixels, GPUs now power AI/ML training, scientific simulation, and video transcoding.

Type	Details
Integrated GPU	Built into CPU die. Shares system RAM. Low power. Suitable for 4K video, light gaming. Examples: Intel Iris Xe, AMD Radeon Graphics.
Discrete GPU	Separate PCIe card with its own VRAM (GDDR6X). High performance. Examples: NVIDIA RTX 4090 (24GB VRAM).
NVIDIA CUDA	NVIDIA's proprietary parallel computing platform. Industry standard for AI/ML (PyTorch, TensorFlow run natively).
AMD OpenCL	Open standard for GPU compute — supported by AMD, Intel, and even NVIDIA. Used in creative tools (Blender).
Vulkan / Metal	Low-level graphics APIs replacing OpenGL. Vulkan = cross-platform, Metal = Apple-only, giving near-bare-metal performance.

2.5 Storage — HDD, SSD, NVMe

Type	Interface	Speed (Read)	Capacity	Best For
HDD	SATA III	~150 MB/s	Up to 20 TB	Bulk storage, backups
SATA SSD	SATA III	~550 MB/s	Up to 8 TB	OS drive (budget)
NVMe SSD	PCIe 3.0 x4	~3,500 MB/s	Up to 8 TB	Fast OS & games
NVMe SSD	PCIe 4.0 x4	~7,000 MB/s	Up to 8 TB	Workstation
NVMe SSD	PCIe 5.0 x4	~14,000 MB/s	Up to 4 TB	High-end servers

■ ***Note:*** *NVMe = Non-Volatile Memory Express — a protocol designed specifically for flash storage, bypassing the legacy AHCI layer used by SATA.*

2.6 Network Card, Sound Card, PCIe Slots & Diagnostic Card

Component	Explanation
Network Card (NIC)	Provides wired (Ethernet) or wireless (Wi-Fi / Bluetooth) connectivity. Modern NICs: 1 GbE onboard, 2.5 GbE on newer boards.
Sound Card	Converts digital audio (PCM) to analogue (DAC) for speakers, and analogue input to digital (ADC) for microphones.
PCIe Slots	PCI Express (PCIe) is the high-speed serial bus. Slots: x1 (1 lane), x4 (4 lanes), x8, x16 (GPU). PCIe 5.0 doubles bandwidth per lane vs PCIe 4.0.
POST Diagnostic Card	A small PCIe/PCI card with a 2-digit hex display. Shows POST codes during boot — invaluable when the system fails to display video.

2.7 Bus Interfaces — SATA, ATA / PATA

Interface	Full Name	Speed	Connector	Status
PATA/ATA	Parallel ATA (IDE)	Up to 133 MB/s	40-pin ribbon	Legacy (1980s-2000s)
SATA I	Serial ATA 1.0	150 MB/s	7-pin data	Obsolete
SATA II	Serial ATA 2.0	300 MB/s	7-pin data	Rare
SATA III	Serial ATA 3.0	600 MB/s	7-pin data	Current standard
eSATA	External SATA	600 MB/s	eSATA port	Mostly replaced by USB3

2.8 Input Devices

Input devices send data into the computer for processing. They convert physical actions or signals into digital form the CPU can read.

Device	How It Works
Keyboard	Generates scan codes per key via a microcontroller (USB HID protocol).
Mouse	Optical sensor captures movement (DPI = dots per inch). Buttons send HID reports.
Touchscreen	Capacitive (smartphone) or resistive (industrial). Multi-touch via TUIO protocol.
Scanner	CCD/CIS sensor converts physical document to digital image (DPI = resolution).
Microphone	Converts sound pressure waves to electrical signal (ADC converts to digital PCM).
Webcam	CMOS sensor array; outputs YUV or MJPEG via USB UVC standard.
Other devices	Joystick, gamepad, light pen, barcode reader, biometric scanner, NFC reader.

2.9 Output Devices

Output devices take processed data from the computer and present it to the user in a human-readable form.

Device	How It Works
Monitor	LCD/IPS/OLED panel. GPU sends DisplayPort/HDMI signals (pixel clock, H/V sync).
Printer	Inkjet (droplets, 1200 DPI) or Laser (electrostatic drum, toner). PCL/PostScript language.
Speaker	DAC (in sound card or USB) converts PCM digital audio to analogue voltage -> coil -> cone.
Projector	DLP (DMD chip) or LCD. Lumens = brightness. Throw ratio = distance/image width.
LED/Screen	HDMI matrix displays, e-ink (bistable - holds image without power), VR headsets.

PART 3 Disk Structure

3.1 Physical Disk Anatomy

A traditional Hard Disk Drive (HDD) contains spinning magnetic platters coated with a ferrous oxide layer. A read/write head floats nanometres above the surface on an air cushion. An SSD has no moving parts — it stores charge in NAND flash cells (SLC, MLC, TLC, QLC). An NVMe drive is physically an M.2 form-factor SSD communicating over PCIe lanes.

HDD Physical Structure

- **Platter** (magnetic disk) — multiple stacked platters per drive
- **Track** — concentric circle on platter surface
- **Sector** — smallest addressable unit (512B traditional / 4096B Advanced Format)
- **Cylinder** — same track number across all platters
- **Cluster** — OS allocation unit = N x sectors (e.g. 4 KB default)
- **Head** — read/write element; one per platter side

SSD/NVMe Structure

```
NAND die -> Plane -> Block (256-512 pages) -> Page (4-16 KB)
Controller - handles wear leveling, ECC, garbage collection
DRAM cache - speeds up random writes (some budget SSDs omit this!)
```

Disk Anatomy Diagram

```
Top View of Platter Side View (Multiple Platters)
+--------------------------+ +------------------------+
| Track 0 (outermost) | | Platter 1 --- Head 0 |
| +------------------+ | | Platter 1 --- Head 1 |
| | Track 1 | | | Platter 2 --- Head 2 |
| | +-------------+ | | | Platter 2 --- Head 3 |
| | | Sector | | | | (Heads move together |
| | | (pie slice) | | | | on shared actuator |
| | +-------------+ | | | arm = Cylinder) |
| +------------------+ | +------------------------+
+--------------------------+
Cluster = group of sectors (smallest unit OS can allocate)
```

3.2 Logical Disk Layout

Regardless of physical media, all disks are logically divided into LBA (Logical Block Addresses). The OS file system then organises blocks into files and directories.

Term	Size	Description
Sector	512 B / 4 KB	Smallest hardware unit
Block	4 KB (typical)	OS logical unit (= cluster on Windows)
Page (SSD)	4-16 KB	Smallest writable flash unit
Extent	Variable	Contiguous set of blocks (EXT4, NTFS)
Inode	128-256 B	Linux metadata structure per file

PART 4 Operating System & Kernel

4.1 OS vs Kernel — The Difference

The Kernel is the core program that runs with full hardware privilege (Ring 0). It manages the CPU, memory, devices, and system calls. The Operating System is the complete package: kernel + shell + file system + utilities + GUI + package manager. Think of the kernel as the engine; the OS is the whole car.

Term	Meaning
Kernel	Lowest-level software; talks directly to hardware; always in memory.
OS	Kernel + user-space programs (shell, GUI, services, libraries).
System Call (syscall)	Interface between user-space programs and the kernel (e.g. read(), write(), fork()).
User Space	Where applications run (Ring 3). Cannot directly touch hardware.
Kernel Space	Ring 0. Full hardware access. A crash here = system crash (kernel panic / BSOD).

4.2 Types of Kernels

Monolithic Kernel

All OS services (file system, drivers, networking) run in kernel space. Fast (no context switching for services) but a bug in any driver can crash the whole OS. Examples: Linux, MS-DOS.

Microkernel

Only the most essential services (IPC, basic scheduling, memory) run in kernel space. Everything else (drivers, file system) runs as user-space servers. More stable — a driver crash does not kill the OS. Examples: Mach, QNX, seL4.

Hybrid Kernel

Monolithic core but some services moved to user space for safety. Best of both worlds in practice. Examples: Windows NT, macOS XNU.

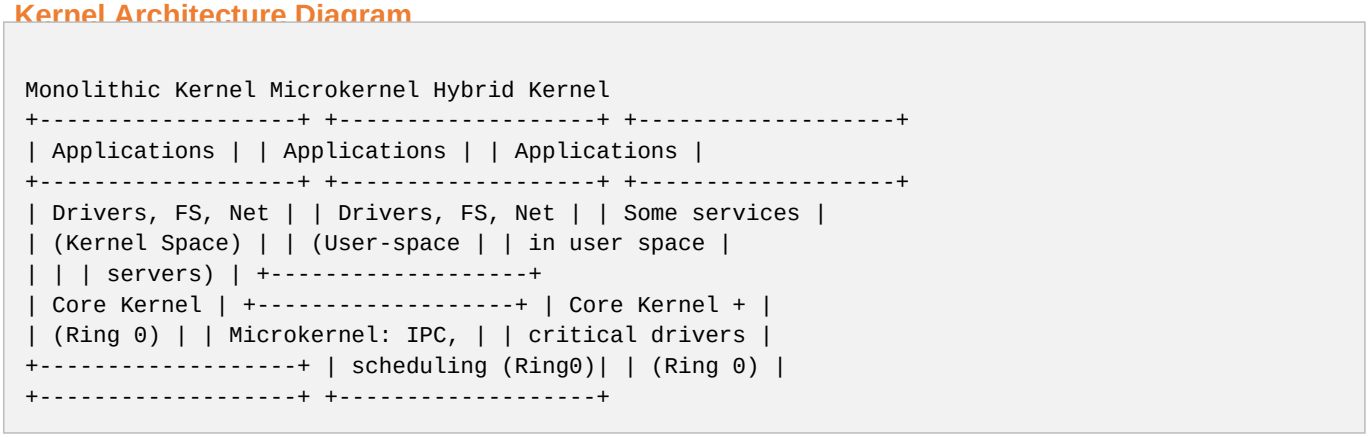
Exokernel

Research kernel — exposes hardware directly to applications, which manage their own abstractions. Maximum performance. Examples: MIT Exokernel, Xok.

Unikernel

Application and OS compiled into a single image for a VM. No user/kernel split. Ultra-fast boot, minimal attack surface. Used in cloud/IoT. Examples: MirageOS, Unikraft.

Kernel Architecture Diagram



4.3 OS Customisation

Linux is fully open-source (GPL licence), allowing deep customisation:

- **Kernel recompilation** — remove unused drivers, add real-time patches (PREEMPT_RT), enable specific CPU optimisations with make menuconfig
- **Custom init systems** — replace systemd with OpenRC, runit, or s6 for faster boot or embedded use
- **Desktop Environment** — GNOME, KDE Plasma, XFCE, i3 (tiling WM), Sway (Wayland)
- **Custom distros** — Build your own with Linux From Scratch (LFS), Buildroot, Yocto (embedded), or Arch Linux
- **Windows** — Limited: themes via third-party tools (Rainmeter), registry tweaks, removing AppX packages. No kernel source access

4.4 Intel x86 Assembly vs ARM Assembly

Assembly language is a human-readable representation of machine code. Each CPU architecture has its own ISA (Instruction Set Architecture). x86 is CISC (Complex Instruction Set Computing); ARM is RISC (Reduced Instruction Set Computing).

```
--- Intel x86-64 (AT&T syntax) --- Add two numbers ---
mov $10, %rax ; rax = 10
mov $20, %rbx ; rbx = 20
add %rbx, %rax ; rax = rax + rbx (30)
ret

--- ARM64 (AArch64) --- Add two numbers ---
mov x0, #10 // x0 = 10
mov x1, #20 // x1 = 20
add x0, x0, x1 // x0 = x0 + x1 (30)
ret
```

Feature	x86 / x86-64	ARM / ARM64
ISA type	CISC	RISC
Instructions	Variable length (1-15B)	Fixed 32-bit (Thumb: 16/32)
Registers	16 GP (RAX-R15)	31 GP (X0-X30) + SP, PC
Memory access	Any instruction	Load/Store only
Power	Higher	Lower (mobile/server)
Found in	PC, Server (x86)	Phone, Mac M-series, AWS Graviton

PART 5 OS Booting Process

5.1 POST — Power-On Self Test

When you press the power button, the CPU starts executing code from a fixed address in ROM (the BIOS/UEFI firmware). The first task is POST:

- Initialise CPU registers and cache
- Test RAM — write and read back patterns to detect faulty cells
- Detect and initialise PCI/PCIe devices (GPU, NIC, storage controllers)
- Check for keyboard, mouse, USB devices
- Report errors via beep codes (legacy) or POST diagnostic display codes
- If POST passes, control transfers to the boot manager (BIOS/UEFI)

5.2 BIOS vs UEFI

Feature	Legacy BIOS	UEFI
Storage	16-bit code in ROM	C code in flash, can be >16 MB
Disk support	MBR only (2 TB max)	GPT (9.4 ZB max)
Boot mode	Real mode (16-bit)	32/64-bit protected mode
Interface	Text only	Full GUI with mouse support
Secure Boot	Not supported	Supported (blocks unauthorised OS)
Network boot	Basic PXE	HTTP/HTTPS boot, IPv6 PXE
Speed	Slower (~30s)	Fast (~3-5s with Fast Boot)

5.3 ACPI Power Tables

ACPI (Advanced Configuration and Power Interface) is a standard that allows the OS to control power management — sleep (S3), hibernate (S4), shutdown (S5), CPU frequency scaling (P-states), and thermal management. The BIOS/UEFI populates ACPI tables in memory (RSDP, XSDT, FADT, MADT, DSDT) which the OS reads on boot. Linux exposes these at **/sys/firmware/acpi/tables/**.

5.4 Bootstrap & Boot Sequence

The bootstrap loader is the tiny program (512 bytes in MBR, or a UEFI .efi file) that loads the full bootloader (GRUB, Windows Boot Manager). The full sequence:

- **1.** Power on -> CPU executes firmware reset vector (0xFFFF0 for BIOS / UEFI flash)
- **2.** POST completed
- **3.** BIOS: reads MBR sector 0 into RAM 0x7C00, jumps to it. / UEFI: reads EFI\BOOT\BOOTx64.EFI from EFI partition
- **4.** Stage-1 bootloader (446 bytes) finds and loads Stage-2 (GRUB core.img or Windows bootmgr)
- **5.** Stage-2 bootloader presents boot menu, loads OS kernel + initrd into RAM
- **6.** Kernel decompresses itself, initialises hardware, mounts root filesystem
- **7.** Kernel launches PID 1 (systemd/init) — all user processes stem from here

5.5 How Booting Happens in Memory (Diagram)

Physical RAM Layout During Boot

Address	Contents
0x00000000	IVT (Interrupt Vector Table) — 1 KB
0x00000400	BIOS Data Area (BDA) — 256 B
0x00007C00	MBR / Stage-1 bootloader — 512 B <-- BIOS jumps here
0x00010000	GRUB Stage-2 / Boot modules
0x00100000 (1 MB mark)	Extended RAM begins
...	Kernel image loaded here (~10-50 MB)
...	initrd (initial RAM disk) (~30-80 MB compressed)
...	Kernel stack, page tables
...	Available for OS + processes
0xFFFFC000	UEFI Runtime Services (remapped)

■ ***Note:*** *After kernel init: BIOS/UEFI memory can be reclaimed by the OS.*

5.6 CPU Code Names

Brand	Code Name	Gen / Year	Process Node
Intel	Raptor Lake	13th Gen 2022	Intel 7 (10nm)
Intel	Meteor Lake	14th Gen 2023	Intel 4 (7nm EUV)
Intel	Arrow Lake	15th Gen 2024	Intel 20A / TSMC N3
AMD	Zen 4 (Raphael)	Ryzen 7000 2022	TSMC 5nm
AMD	Zen 5 (Granite Ridge)	Ryzen 9000 2024	TSMC 4nm
ARM	Cortex-X925	2024 flagship	TSMC 3nm
Apple	M4 (Donan)	2024	TSMC 3nm

PART 6 Operating Systems Survey

6.1 Windows Family

Version	Year	Kernel	Notes
Windows NT 4	1996	NT 4.0	First modern NT kernel
Windows XP	2001	NT 5.1	Longest-used consumer OS
Windows 7	2009	NT 6.1	Most beloved Windows
Windows 10	2015	NT 10.0	WaaS model (updates as service)
Windows 11	2021	NT 10.0	TPM 2.0 required, Android subsystem
Windows Server	2022	NT 10.0	Azure-integrated, Secured-core

6.2 Linux Distributions

Linux is a kernel — distributions (distros) bundle the kernel with package managers, desktop environments, and tools for specific purposes.

Distro	Base	Package Mgr	Target Use
Ubuntu	Debian	apt/snap	Desktop, cloud, beginners
Debian	—	apt	Stable server, base for others
Kali Linux	Debian	apt	Penetration testing, security
Parrot OS	Debian	apt	Security + privacy focused
Red Hat (RHEL)	Fedora	dnf/rpm	Enterprise server, paid support
Fedora	Red Hat	dnf	Cutting-edge, developer desktop
Arch Linux	—	pacman	DIY, rolling release, advanced users
Raspbian (PiOS)	Debian	apt	Raspberry Pi single-board computer

6.3 Unix / Solaris / OpenIndiana

Solaris (Sun Microsystems, now Oracle) introduced ZFS, Zones (containers), DTrace, and SMF. OpenIndiana is the open-source illumos-based fork maintained by the community after Oracle closed Solaris. These systems excel in enterprise storage and high-availability environments. FreeBSD and OpenBSD are other prominent UNIX-like systems known for security and clean design.

6.4 Embedded — Raspbian (Raspberry Pi OS)

Raspberry Pi OS (formerly Raspbian) is a Debian-based distro optimised for the ARM-based Raspberry Pi hardware. It ships with Python, GPIO libraries, and a lightweight LXDE desktop. Used for education, IoT, home automation, retro gaming (RetroPie), and media centres (LibreELEC / Kodi).

6.5 SDK & NDK

Component	Explanation
SDK — Software Development Kit	A collection of tools, libraries, headers, and sample code that allows developers to write software for a specific platform (e.g. Android SDK, Windows SDK).
NDK — Native Development Kit	Allows writing performance-critical code in C/C++ for platforms that primarily use higher-level languages. The Android NDK lets apps call native libraries via JNI.

PART 7 Partitioning — MBR & GPT

7.1 MBR — Master Boot Record

MBR is the legacy partitioning scheme, introduced with IBM PC DOS in 1983. The first 512-byte sector of the disk contains: 446 bytes of bootstrap code, 64 bytes of partition table (4 entries x 16 bytes), and a 2-byte signature (0x55AA).

MBR Sector Layout (512 bytes total)

```
[0x000 - 0x1BD] Bootstrap code (446 bytes) <-- executes on boot
[0x1BE - 0x1CD] Partition Entry 1 (16 bytes)
[0x1CE - 0x1DD] Partition Entry 2 (16 bytes)
[0x1DE - 0x1ED] Partition Entry 3 (16 bytes)
[0x1EE - 0x1FD] Partition Entry 4 (16 bytes)
[0x1FE - 0x1FF] Boot signature: 0x55 0xAA
```

Limitations:

- Max 4 primary partitions (workaround: extended + logical)
- Max disk size: 2 TB (uses 32-bit LBA)

7.2 GPT — GUID Partition Table

GPT is part of the UEFI standard, replacing MBR. It uses 64-bit LBA addresses and stores multiple backup copies of the partition table at both ends of the disk.

GPT Disk Layout

```
LBA 0: Protective MBR (keeps legacy tools from misidentifying the disk)
LBA 1: Primary GPT Header (92 bytes: disk GUID, # partitions, CRC32)
LBA 2-33: Partition entries (128 entries x 128 bytes each)
LBA 34+: Partition data
Last 33 LBAs: Backup partition entries
Last LBA: Backup GPT header
```

Advantages:

- Up to 128 partitions (Windows), unlimited on Linux
- Max disk size: 9.4 ZB (zettabytes)
- Redundant headers — survives partial corruption
- Each partition has a GUID, name (36-char Unicode), and attribute flags

7.3 MBR vs GPT Comparison

Feature	MBR	GPT
Max disk size	2 TB	9.4 ZB
Max partitions	4 primary	128 (Windows) / unlimited (Linux)
Backup	None	Redundant header at end of disk
Firmware	BIOS + UEFI	UEFI recommended
Boot code	In sector 0	In EFI System Partition (FAT32)
Windows support	XP and later	Vista 64-bit and later
Linux support	Full	Full (parted, gdisk)
Integrity check	None	CRC32 checksum

PART 8 File Systems

8.1 FAT — File Allocation Table

FAT (FAT12 / FAT16 / FAT32 / exFAT) is the oldest and most compatible file system. It uses a table of cluster numbers to track file locations. FAT32 has a 4 GB file size limit. exFAT (2006) removes this limit — used on SD cards and USB drives for cross-platform compatibility. No journalling, no permissions — fragile but universal.

Key	Detail
FAT32	Max file: 4 GB, max volume: 2 TB, cluster: 4-32 KB
exFAT	Max file: 128 PB, max volume: 128 PB, used in SDXC/SDUC
Use case	USB drives, SD cards, embedded devices, game consoles

8.2 NTFS — New Technology File System

NTFS is the default Windows file system since NT 3.1. It uses a Master File Table (MFT) — a database where every file is a record. Features: journalling (recovers from crashes), ACL-based permissions, ADS (Alternate Data Streams), compression, encryption (EFS), hard links, junctions, symbolic links, volume shadow copies.

Key	Detail
MFT	Each file = 1 KB record; small files stored inside MFT itself
Journalling	Transaction log ($LogFile) — ensures metadata consistency
Compression	Per-file LZNT1; useful for archival files
Permissions	Full ACL (Discretionary ACL + System ACL)
Linux	Read/write via ntfs-3g or kernel ntfs3 driver

8.3 ZFS — Zettabyte File System

ZFS (Sun Microsystems, 2004) is a combined file system and volume manager. It treats all storage as a pool. Features: copy-on-write (COW) transactions, always-consistent on-disk state, end-to-end checksumming (SHA-256/Fletcher), inline compression (LZ4/ZSTD), deduplication, snapshots, clones, RAIDZ.

Key	Detail
Pool (zpool)	Collection of vdevs (virtual devices — mirrors, RAIDZ)
Snapshots	Instant, zero-cost point-in-time copies (COW)
Checksums	Detects and auto-heals silent data corruption (bit rot)
ARC cache	Adaptive Replacement Cache — intelligent RAM cache
Platforms	OpenZFS: FreeBSD, Linux (zfs-linux), macOS (OpenZFS on OS X)

8.4 EXT4 — Fourth Extended File System

EXT4 is the default Linux file system, introduced in 2008. It is a journalled file system with extents (contiguous block ranges per file), delayed allocation (reduces fragmentation), and nanosecond timestamps. Max file size: 16 TB. Max volume size: 1 EB.

Key	Detail
Journalling	Metadata journal prevents corruption (journal=data for full safety)
Extents	A single extent can map up to 128 MB contiguously — fast large file I/O
Inodes	Fixed at format time (use -N to increase); each file needs one
fsck	e2fsck checks and repairs; online resizing with resize2fs
Successor	BTRFS and F2FS being adopted; BTRFS is default in openSUSE/Fedora

8.5 NFS — Network File System

NFS (Sun Microsystems, 1984) allows a client to mount a remote directory over a network as if it were local. The NFS server exports directories; clients mount them. NFSv4 added stateful operations, strong security (Kerberos), and ACLs. Used extensively in data centres, HPC clusters, and NAS devices.

Key	Detail
NFSv3	Stateless, UDP/TCP, no built-in auth — relies on IP trust
NFSv4	Stateful, TCP only, Kerberos auth, compound operations
Mount	mount -t nfs server:/export /mnt/share
vs SMB	Samba/SMB is the Windows equivalent — used in mixed environments
vs iSCSI	iSCSI presents a block device over network; NFS presents a file system

PART 9 Bootloader & GRUB

9.1 What is a Bootloader?

A bootloader is a small program that runs before the OS. Its job: initialise minimum hardware, locate the OS kernel on disk, load it into RAM, and hand over control. Without a bootloader, the CPU has no way to find and start your OS.

Component	Description
Stage 1	Lives in MBR (446 bytes) or UEFI EFI partition. Loads Stage 2.
Stage 2	Full bootloader. Reads config file, shows menu, decompresses kernel.
Windows BM	Windows Boot Manager (bootmgr / winload.efi) — proprietary, UEFI-native.
GRUB2	GRand Unified Bootloader v2 — universal, open-source, supports EFI + BIOS.
LILO	Linux Loader — legacy, no menu, still used in some embedded systems.
systemd-boot	Minimal UEFI-only bootloader, part of systemd. Fast, simple config.
rEFInd	Graphical UEFI boot manager — auto-detects all OSes on all disks.

9.2 GRUB2 — GRand Unified Bootloader (Open Source)

GRUB2 is the standard bootloader for virtually all Linux distributions. It supports BIOS + UEFI, reads EXT4/BTRFS/ZFS/FAT/NTFS file systems, loads Linux/Windows/BSD kernels, and supports chainloading. Configuration lives at **/etc/default/grub** and **/etc/grub.d/** (script-generated).

GRUB2 Boot Sequence

```
BIOS: MBR stage1 -> /boot/grub/i386-pc/core.img -> grub.cfg
UEFI: /EFI/ubuntu/grubx64.efi -> /boot/grub/grub.cfg
```

9.3 GRUB Configuration

/etc/default/grub (key settings):

```
GRUB_DEFAULT=0 # Default menu entry (0-indexed)
GRUB_TIMEOUT=5 # Seconds to show menu
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash" # Kernel parameters
GRUB_DISABLE_OS_PROBER=false # Detect other OSes (Windows etc.)
```

After editing, always run:

```
sudo update-grub
# Regenerates /boot/grub/grub.cfg

sudo grub-install /dev/sda
# Reinstalls GRUB to disk (BIOS)

sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi
# Reinstalls GRUB for UEFI
```

GRUB Manual Entry Example

```
# /boot/grub/grub.cfg – manual entry for Ubuntu:
menuentry 'Ubuntu 24.04 LTS' {
insmod gzio
insmod part_gpt
insmod ext2
set root='hd0,gpt2'
linux /boot/vmlinuz-6.8.0 root=/dev/sda2 ro quiet splash
initrd /boot/initrd.img-6.8.0
}

# Chain-load Windows from GRUB:
menuentry 'Windows 11' {
insmod part_gpt
insmod fat
set root='hd0,gpt1'
chainloader /EFI/Microsoft/Boot/bootmgfw.efi
}
```

PART 10 Multi-OS Setup

10.1 Dual-Boot Concepts

Dual-booting means installing two or more operating systems on one machine, each on separate partitions. GRUB (or Windows Boot Manager) presents a menu at startup. Key rules:

- **Install Windows first** — Windows overwrites the MBR/EFI without asking. Install Linux second so GRUB can detect Windows
- **Separate partitions** — Each OS needs its own root partition. A shared /home or data partition can use NTFS or EXT4
- **Shared EFI partition** — On UEFI systems, all OSes share the single EFI System Partition (FAT32, ~512 MB)
- **Time zone conflict** — Windows stores local time in RTC; Linux stores UTC. Fix on Linux: timedatectl set-local-rtc 1
- **BitLocker warning** — Dual-booting may trigger BitLocker recovery on Windows. Suspend BitLocker before changing boot config

Quick Dual-Boot Setup (Windows 11 + Ubuntu)

- **1.** Install Windows 11 on Disk 0, Partition 1 (e.g. 100 GB)
- **2.** Leave unallocated space for Ubuntu (e.g. 80 GB)
- **3.** Boot Ubuntu installer USB -> choose 'Something else' (manual)
- **4.** Create partitions in free space:

```
/boot/efi -> FAT32, 512 MB (use existing EFI if present!)
/ -> EXT4, 40 GB
/home -> EXT4, remaining
swap -> swap, = RAM GB
```

- **5.** Finish install — GRUB installs to EFI partition
- **6.** Reboot -> GRUB menu shows Ubuntu + Windows
- **7.** If Windows is missing: sudo os-prober && sudo update-grub

10.2 Virtual Machines (VMs)

A hypervisor creates isolated virtual machines, each running a full OS. No rebooting needed — run multiple OSes simultaneously.

Hypervisor	Type	Platforms	Notes
VMware Workstation	Type 2	Windows/Linux	Industry standard, snapshots
VirtualBox	Type 2	Win/Linux/macOS	Free, open-source
Hyper-V	Type 1	Windows 10/11 Pro	Built into Windows, WSL2 backend
QEMU/KVM	Type 1	Linux	Bare-metal performance, cloud standard
Parallels	Type 2	macOS	Best VM for Apple Silicon Mac
VMware Fusion	Type 2	macOS	Professional macOS VM solution

10.3 WSL — Windows Subsystem for Linux

WSL lets you run a full Linux kernel (WSL2) inside Windows without a VM or dual-boot. WSL2 uses a lightweight Hyper-V VM with a real Linux kernel. You get native Linux syscalls, full bash shell, GUI apps (WSLg), GPU pass-through (CUDA/OpenCL), and full integration with Windows files.

WSL2 — Common Commands

```
# Install WSL2 (PowerShell as Administrator):
wsl --install

# Install a specific distro:
wsl --install -d Ubuntu-24.04
wsl --install -d kali-linux

# List installed distros:
wsl --list --verbose

# Launch & use Linux from PowerShell:
wsl
sudo apt update && sudo apt upgrade -y

# Run Linux command from Windows:
wsl ls -la /home
wsl python3 script.py
```

■ ***Note:*** *WSL2 files live at* ***\\wsl$\Ubuntu*** *in Windows Explorer. Windows drives are mounted at* ***/mnt/c/, /mnt/d/*** *etc. inside WSL.*

Summary: Choosing Your Multi-OS Strategy

Scenario	Recommended Approach
Testing Linux, keep Windows	WSL2 — no rebooting, full Linux environment
Need full Linux performance	Dual-boot — bare-metal speed, separate OS
Run multiple OSes simultaneously	VM (VirtualBox / QEMU-KVM)
Development across platforms	WSL2 + Docker for Linux, VM for macOS testing
Security / Forensics lab	VM with snapshots (easy reset to clean state)
Embedded / Raspberry Pi work	Native Linux on Pi or QEMU ARM emulation

---